```
/*
 * File:   main.cpp
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 28 de junio de 2023, 02:21 PM
 */

#include <iostream>
#include <fstream>
#include <iomanip>

using namespace std;

#include "UsuarioConElLibro.h"
#include "Libro.h"
#include "LibroEnSuPoder.h"
#include "Usuario.h"

#include "funciones.h"

#define TAM_MAX_LIBROS 50
#define TAM_MAX_USUARIOS 100

int main(int argc, char** argv) {
    struct Libro arrLibros[TAM_MAX_LIBROS];
    struct Usuario arrUsuarios[TAM_MAX_USUARIOS];
    int cantLibros,cantUsuarios;


    /*si sería dinámico
    struct Libro *arrLibros;
    arrLibros= new struct Libro[TAM_MAX_LIBROS];*/
    cargarLibros(arrLibros,cantLibros);
    imprimirLibros(arrLibros,cantLibros,"librosInicial.txt");
    cargarUsuarios(arrUsuarios,cantUsuarios);
    imprimirUsuarios(arrUsuarios,cantUsuarios,"usuariosInicial.txt");
    procesarDevoluciones(arrLibros,cantLibros,arrUsuarios,cantUsuarios);
    imprimirLibros(arrLibros,cantLibros,"librosActualizado.txt");
    imprimirUsuarios(arrUsuarios,cantUsuarios,"usuariosActualizado.txt");

    return 0;
}

/*
 * File:   UsuarioConElLibro.h
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 29 de junio de 2023, 12:15 PM
 */

#ifndef USUARIOCONELLIBRO_H
#define USUARIOCONELLIBRO_H

struct UsuarioConElLibro{
    int dni;
    int fechaDeDevolucion;
};

#endif /* USUARIOCONELLIBRO_H */

/*
 * File:   LibroensuPoder.h
 * Author: Silvia
 * Silvia Vargas Cáceres
 *
 * Created on 28 de junio de 2023, 02:21 PM
 */
```

```
70
71  #ifndef LIBROENSUPODER_H
72  #define LIBROENSUPODER_H
73
74  struct LibroEnSuPoder{
75      char *codLibro;
76  };
77
78  #endif /* LIBROENSUPODER_H */
79
80  /*
81   * File:   Libro.h
82   * Author: Silvia
83   * Silvia Vargas Cáceres
84   *
85   * Created on 28 de junio de 2023, 02:24 PM
86   */
87
88  #ifndef LIBRO_H
89  #define LIBRO_H
90
91  struct Libro{
92      char *codigo;
93      char *titulo;
94      char *autor;
95      int cantidad;
96      double precio;
97      struct UsuarioConElLibro *usuarios;
98      int cantPrestados;
99  };
100
101 #endif /* LIBRO_H */
102
103
104 /*
105  * File:   Usuario.h
106  * Author: Silvia
107  * Silvia Vargas Cáceres
108  *
109  * Created on 28 de junio de 2023, 02:22 PM
110  */
111
112 #ifndef USUARIO_H
113 #define USUARIO_H
114
115 struct Usuario{
116     int dni;
117     char *nombre;
118     char categoria;
119     double calificacion;
120     struct LibroEnSuPoder *libros;
121     int cantLibros;
122 };
123
124 #endif /* USUARIO_H */
125
126 /*
127  * File:   funciones.h
128  * Author: Silvia
129  * Silvia Vargas Cáceres
130  *
131  * Created on 28 de junio de 2023, 02:31 PM
132  */
133
134 #ifndef FUNCIONES_H
135 #define FUNCIONES_H
136
137 void cargarLibros(struct Libro *,int &);
138 void imprimirLibros(struct Libro *,int ,const char *);
```

```
139 void cargarUsuarios(struct Usuario *,int &);
140 void imprimirUsuarios(struct Usuario *,int &,const char *);
141 void procesarDevoluciones(struct Libro *,int ,struct Usuario *,int );
142 void registrarDevolucion(int ,int ,char *,struct Libro *,int ,struct Usuario *,
143         int );
144 int buscarUsuario(int ,struct Usuario *,int );
145 int buscarLibro(char *,struct Libro *,int );
146 char *leeCadenaExactaDelim(ifstream &,char );
147 void escribirSimbolo(ofstream &,char ,int );
148
149 #endif /* FUNCIONES_H */
150
151
152 /*
153  * File:   funciones.cpp
154  * Author: Silvia
155  * Silvia Vargas Cáceres
156  *
157  * Created on 28 de junio de 2023, 02:31 PM
158  */
159
160 #include <iostream>
161 #include <fstream>
162 #include <iomanip>
163 #include <cstring>
164
165 using namespace std;
166
167 #include "UsuarioConElLibro.h"
168 #include "Libro.h"
169 #include "LibroEnSuPoder.h"
170 #include "Usuario.h"
171
172 #include "funciones.h"
173
174 #define MAX_PRESTAMOS 6
175 #define NO_ENCONTRADO -1
176 #define TAM_LINEA 110
177
178 void cargarLibros(struct Libro *arrLibros,int &cantLibros){
179     ifstream archLibros("libros.csv",ios::in);
180     if (not archLibros.is_open()){
181         cout<<"El archivo libros.csv no se pudo abrir"<<endl;
182         exit(1);
183     }
184     char *codLibro,*titulo,*autor,car;
185     int cant,dni,dd,mm,aa,i;
186     double precio;
187     cantLibros=0;
188     while(true){
189         codLibro=leeCadenaExactaDelim(archLibros,',');
190         if (codLibro==nullptr) break;
191         titulo=leeCadenaExactaDelim(archLibros,',');
192         autor=leeCadenaExactaDelim(archLibros,',');
193         archLibros>>cant;
194         archLibros.get();
195         archLibros>>precio;
196         archLibros.get();
197         arrLibros[cantLibros].codigo=codLibro;
198         arrLibros[cantLibros].titulo=titulo;
199         arrLibros[cantLibros].autor=autor;
200         arrLibros[cantLibros].cantidad=cant;
201         arrLibros[cantLibros].precio=precio;
202         arrLibros[cantLibros].usuarios=new struct UsuarioConElLibro[cant];
203         i=0;
204         while(true){
205             archLibros>>dni>>car>>dd>>car>>mm>>car>>aa;
206             arrLibros[cantLibros].usuarios[i].dni=dni;
207             arrLibros[cantLibros].usuarios[i].fechaDeDevolucion=aa*1000+mm*100+dd;
```

```
208             i++;
209             if (archLibros.get()=='\n') break;
210         }
211         arrLibros[cantLibros].cantPrestados=i;
212         cantLibros++;
213     }
214 }
215
216 void imprimirLibros(struct Libro *arrLibros,int cantLibros,const char *nombArch){
217     ofstream archReporte(nombArch,ios::out);
218     if (not archReporte.is_open()){
219         cout<<"El archivo "<<nombArch<< " no se pudo abrir"<<endl;
220         exit(1);
221     }
222     archReporte<<setprecision(2)<<fixed;
223     archReporte<<"LISTA DE LIBROS"<<endl;
224     archReporte<<"Codigo"<<setw(25)<<"Titulo"<<setw(25)<<"Autor"<<setw(30)<<
225             "Cantidad"<<setw(9)<<"Precio"<<setw(10)<<"Prestados"<<endl;
226     for (int i=0;i<cantLibros;i++){
227         archReporte<<left<<setw(10)<<arrLibros[i].codigo<<setw(40)<<arrLibros[i].titulo;
228         archReporte<<setw(27)<<arrLibros[i].autor<<right;
229         archReporte<<setw(6)<<arrLibros[i].cantidad<<setw(12)<<arrLibros[i].precio;
230         archReporte<<setw(6)<<arrLibros[i].cantPrestados<<endl;
231         archReporte<<"Prestamos"<<endl;
232         archReporte<<"DNI"<<setw(15)<<"Fecha"<<endl;
233         for (int j=0;j<arrLibros[i].cantPrestados;j++){
234
                archReporte<<setw(10)<<setfill('0')<<setw(8)<<arrLibros[i].usuarios[j].dni<<s
                etfill(' ');
235             archReporte<<setw(10)<<arrLibros[i].usuarios[j].fechaDeDevolucion<<endl;
236         }
237     }
238 }
239
240 void cargarUsuarios(struct Usuario *arrUsuarios,int &cantUsuarios){
241     ifstream archUsuarios("usuarios.csv",ios::in);
242     if (not archUsuarios.is_open()){
243         cout<<"El archivo usuarios.csv no se pudo abrir"<<endl;
244         exit(1);
245     }
246     int dni,dd,mm,aa,i;
247     char *nombre,categoria,car,codLibro[8];
248     double calificacion;
249     cantUsuarios=0;
250     while(true){
251         archUsuarios>>dni;
252         if (archUsuarios.eof()) break;
253         archUsuarios.get();
254         nombre=leeCadenaExactaDelim(archUsuarios,',');
255         //arrUsuarios[cantUsuarios].nombre=leeCadenaExactaDelim(archUsuarios,',');
256         archUsuarios>>categoria;
257         archUsuarios.get();
258         archUsuarios>>calificacion;
259         archUsuarios.get();
260         arrUsuarios[cantUsuarios].dni=dni;
261         arrUsuarios[cantUsuarios].nombre=nombre;
262         arrUsuarios[cantUsuarios].categoria=categoria;
263         arrUsuarios[cantUsuarios].calificacion=calificacion;
264         arrUsuarios[cantUsuarios].libros=new struct LibroEnSuPoder[MAX_PRESTAMOS];
265         i=0;
266         while(true){
267             if (i<MAX_PRESTAMOS){
268                 archUsuarios>>codLibro;
269                 arrUsuarios[cantUsuarios].libros[i].codLibro=new
                    char[strlen(codLibro)+1];
270                 strcpy(arrUsuarios[cantUsuarios].libros[i].codLibro,codLibro);
271                  i++;
272                 if (archUsuarios.get()=='\n') break;
273             }
```

```
274             else
275                 while(archUsuarios.get()!='\n');
276         }
277         arrUsuarios[cantUsuarios].cantLibros=i;
278         cantUsuarios++;
279     }
280 }
281
282 void imprimirUsuarios(struct Usuario *arrUsuarios,int &cantUsuarios,const char
    *nombArch){
283     ofstream archReporte(nombArch,ios::out);
284     if (not archReporte.is_open()){
285         cout<<"El archivo "<<nombArch<< " no se pudo abrir"<<endl;
286         exit(1);
287     }
288     archReporte<<setprecision(2)<<fixed;
289     archReporte<<"LISTA DE USUARIOS"<<endl;
290     archReporte<<"DNI"<<setw(20)<<"Nombre"<<setw(40)<<"Categoria"<<setw(15)
291             <<"Calificacion"<<setw(12)<<"Cantidad"<<endl;
292     for (int i=0;i<cantUsuarios;i++){
293         archReporte<<setfill('0')<<setw(8)<<arrUsuarios[i].dni<<setfill(' ')<<"  ";
294         archReporte<<left<<setw(45)<<arrUsuarios[i].nombre<<right;
295
        archReporte<<setw(3)<<arrUsuarios[i].categoria<<setw(17)<<arrUsuarios[i].calificacion
296         <<setw(11)<<arrUsuarios[i].cantLibros<<endl;
297         archReporte<<"Libros"<<endl;
298         for (int j=0;j<arrUsuarios[i].cantLibros;j++)
299             archReporte<<setw(10)<<arrUsuarios[i].libros[j].codLibro<<endl;
300     }
301 }
302
303 void procesarDevoluciones(struct Libro *arrLibros,int cantLibros,
304         struct Usuario *arrUsuarios,int cantUsuarios){
305     ifstream archDevoluciones("DevolucionDeLibros.txt",ios::in);
306     if (not archDevoluciones.is_open()){
307         cout<<"El archivo DevolucionDeLibros.txt no se pudo abrir"<<endl;
308         exit(1);
309     }
310     int dd,mm,aa,fechaDev,dni;
311     char car,codLibro[8];
312     archDevoluciones>>dd>>car>>mm>>car>>aa;
313     fechaDev=aa*1000+mm*100+dd;
314     while(true){
315         archDevoluciones>>dni;
316         if (archDevoluciones.eof()) break;
317         archDevoluciones>>codLibro;
318
        registrarDevolucion(fechaDev,dni,codLibro,arrLibros,cantLibros,arrUsuarios,cantUsuarios);
319     }
320 }
321
322 void registrarDevolucion(int fechaDev,int dni,char *codLibro, struct Libro *arrLibros,
323         int cantLibros,struct Usuario *arrUsuarios,int cantUsuarios){
324     double calif=0;
325     int posLibro,posUsuario;
326     posUsuario=buscarUsuario(dni,arrUsuarios,cantUsuarios);
327     posLibro=buscarLibro(codLibro,arrLibros,cantLibros);
328     if (posUsuario!=NO_ENCONTRADO && posLibro!=NO_ENCONTRADO){
329         for (int i=0;i<arrLibros[posLibro].cantPrestados;i++)
330             if (arrLibros[posLibro].usuarios[i].dni==dni){
331                 if (arrLibros[posLibro].usuarios[i].fechaDeDevolucion<fechaDev)
332                     calif=arrUsuarios[posUsuario].calificacion;
333                 int cant=arrLibros[posLibro].cantPrestados;
334                 arrLibros[posLibro].usuarios[i]=arrLibros[posLibro].usuarios[cant-1];
335                 delete arrLibros[posLibro].usuarios[cant-1];
336                 arrLibros[posLibro].cantPrestados--;
337             }
```

```
338          for (int i=0;i<arrUsuarios[posUsuario].cantLibros;i++)
339              if (strcmp(codLibro,arrUsuarios[posUsuario].libros[i].codLibro)==0){
340                  int cant=arrUsuarios[posUsuario].cantLibros;
341                  arrUsuarios[posUsuario].libros[i]=arrUsuarios[posUsuario].libros[cant-1];
342                  delete arrUsuarios[posUsuario].libros[cant-1];
343                  arrUsuarios[posUsuario].cantLibros--;
344              }
345          if (calif>0){
346              if (arrUsuarios[posUsuario].categoria=='D')
347                  arrUsuarios[posUsuario].calificacion-=calif*0.2;
348              else
349                  if (arrUsuarios[posUsuario].categoria=='V')
350                      arrUsuarios[posUsuario].calificacion-=calif*0.15;
351                  else
352                      arrUsuarios[posUsuario].calificacion-=calif*0.1;
353              if (arrUsuarios[posUsuario].calificacion<0)
354                  arrUsuarios[posUsuario].calificacion=0;
355          }
356      }
357  }
358
359  int buscarUsuario(int dni,struct Usuario *arrUsuarios,int cantUsuarios){
360      for (int i=0;i<cantUsuarios;i++)
361          if (arrUsuarios[i].dni==dni) return i;
362      return NO_ENCONTRADO;
363  }
364
365  int buscarLibro(char *codLibro,struct Libro *arrLibros,int cantLibros){
366      for (int i=0;i<cantLibros;i++)
367          if (strcmp(arrLibros[i].codigo,codLibro)==0) return i;
368      return NO_ENCONTRADO;
369  }
370
371  char *leeCadenaExactaDelim(ifstream &archivo,char delim){
372      char cadena[500],*ptrCadena;
373      int longitud;
374      archivo.getline(cadena,500,delim);
375      if (archivo.eof()) return nullptr;
376      longitud=strlen(cadena);
377      ptrCadena=new char[longitud+1];
378      strcpy(ptrCadena,cadena);
379      return ptrCadena;
380  }
381
382  void escribirSimbolo(ofstream &archivo,char car,int cantCar){
383      for (int i=0;i<cantCar;i++)
384          archivo.put(car);
385      archivo<<endl;
386  }
387
388
```